Design patterns and backend architecture

Chapter 4: Architecture patterns and when to choose them

Practice designs, not claims about an employer.

Code examples are illustrative fragments, not complete runnable applications.

4. Architecture patterns and when to choose them

1. Layered architecture [High]

Plain English: Group work by role, such as API, application service, business model, and persistence. A request moves through clear responsibilities.

Professional answer: "Layering helps keep transport, business rules, and data access understandable. In a Spring service, the controller handles HTTP, the application service coordinates a use case, and repositories persist state. I watch for business rules leaking into controllers or repositories."

Use when: A service has a straightforward workflow. **Cost:** Too many pass-through layers; an anemic domain model if every rule is pushed into one large service class.

Demonstration: `PaymentController` parses HTTP and returns an API response; `SubmitPaymentService` authenticates business intent and coordinates the use case; `Payment` enforces valid state transitions; `PaymentRepository` reads/writes data.

Why layering: It prevents controllers from becoming business engines and repositories from deciding payment policy. **Proof:** A fee-rule change can be tested without an HTTP server; a request-shape change does not alter domain transitions. **Tradeoff:** Avoid controller → service → service → repository pass-through chains that do not add a meaningful boundary.

Code anchor (illustrative fragment):

```
@RestController
class PaymentController {
    private final SubmitPayment useCase;
    @PostMapping("/payments")
    Receipt submit(@RequestBody PaymentRequest req) {
        return useCase.submit(req.toCommand());
    }
}
// Controller: HTTP; use case: workflow; repository: storage.
```

Verify it: A policy change should not require editing HTTP mapping or SQL. Avoid layers that do nothing beyond forwarding.

2. Hexagonal architecture / ports and adapters [High]

Plain English: Put core business logic in the center. Define ports for what it needs and adapters for HTTP, databases, queues, or vendor systems. The core should not need to know the vendor's API.

Professional answer: "I use ports and adapters when external integrations or storage may change, or when business rules need isolated tests. The application owns a business-oriented port, and an adapter translates to the technology. I do not add a separate interface and adapter for every trivial class."

Example: `SubmitPaymentUseCase` depends on `PaymentStore` and `ClearingGateway`. REST and batch entry points call the same use case. DB2 and provider adapters implement the ports. **Cost:** More boundary code and mapping. See AWS hexagonal architecture guidance.

Demonstration: Two entry adapters, REST and batch-file import, both invoke `SubmitPaymentUseCase`. The use case depends on `PaymentStore` and `ClearingGateway` ports. `JdbcPaymentAdapter` and `ProviderHttpAdapter` implement those ports; neither transport leaks into fee and authorization rules.

Why hexagonal: The application owns stable capabilities while technologies sit outside. **Proof:** The same contract tests run against a fake store and the actual JDBC adapter; changing REST to batch input leaves the core use case unchanged. **Tradeoff:** Boundaries and mapping require code; a small CRUD endpoint may not need all those ports.

Code anchor (illustrative fragment):

```
// Owned by the application core:
interface ClearingPort { GatewayResult clear(Payment p); }
final class SubmitPayment {
    private final ClearingPort clearing;
    SubmitPayment(ClearingPort clearing) {
        this.clearing = clearing;
    }
}
// ProviderSdkAdapter implements ClearingPort at the edge.
```

Verify it: Instantiate the use case in a test with an in-memory port and no provider SDK. The dependency points inward.

3. Modular monolith [High]

Plain English: One deployable application can still have strong internal modules, such as identity, payment, and reconciliation. Each module owns its rules and communicates through clear interfaces.

Professional answer: "I consider a modular monolith when teams need clean boundaries but separate deployments and distributed failure handling are not justified. It is a useful starting point because local transactions and debugging are simpler. I keep module ownership explicit so future extraction remains possible."

Cost: Shared deployment and runtime; boundaries can erode if every module reads another's tables. A module is not isolated merely because it has a package name.

Demonstration: One Spring deployment contains `identity`, `payments`, and `reconciliation` modules. Reconciliation calls an exported `payments` query interface instead of reading payment tables directly; module tests reject forbidden imports.

Why modular monolith: Teams gain explicit ownership without network latency and distributed transactions for every interaction. **Proof:** A payment schema refactor does not require edits in the reconciliation module; module dependency rules are enforced in CI. **Tradeoff:** The application still deploys and fails as one process. Extract a service only when the independent deployment or scaling need is proven.

Code anchor (illustrative fragment):

```java
// identity module exposes an application-level interface:
interface IdentityLookup {
    Principal findAuthorized(UserId actor);
}
// payments module depends on IdentityLookup,
// not identity's private tables or persistence classes.
final class SubmitPayment {
    SubmitPayment(IdentityLookup identities) { /* ... */ }
}
```

Verify it: Add a module-boundary test: payments must not import identity.persistence.* or query its tables directly.

4. Microservices with owned data [High]

Plain English: Independent services own capabilities and, ideally, control their own data. They communicate through APIs or events. This helps independent change but creates network, consistency, testing, and operations costs.

Professional answer: "I choose service boundaries around business capabilities and team ownership, not around small classes. I account for API contracts, data ownership, observability, deployment, and partial failure. I do not split a cohesive workflow simply because microservices are popular."

Caution: Database-per-service means one service controls access to its data; it does not require a separate database server for every service. Cross-service reports need API composition, materialized views, or events. See database-per-service pattern.

Demonstration: The Payment service owns instructions; Reconciliation owns matching runs; Identity owns user access data. Reconciliation consumes `PaymentSettled` events or a supported Payment API rather than querying Payment tables directly.

Why microservices: Separate teams may need independent releases, capacity, and fault isolation. **Proof:** The reconciliation deploy leaves Payment unchanged; a reconciliation outage does not corrupt accepted payments, and lag is monitored. **Tradeoff:** Cross-service reports require projections/APIs, events may be delayed, and failure recovery is harder. A split that causes constant synchronous calls may be worse than a modular monolith.

Code anchor (illustrative fragment):

```
Payment Service --HTTP/event--> Identity Service
      |                              |
Payment-owned tables         Identity-owned tables

// No cross-service SQL join or shared-table writes.
// Payment may cache a permitted identity projection.
```

Verify it: Stop Identity Service: Payment must have a defined timeout or degraded response. Teams must own API versioning and data boundaries.

5. Synchronous request/response [High]

Plain English: A caller sends a request and waits for an answer. It is simple when the answer is needed now. The caller's response time and availability depend on the downstream path.

Professional answer: "I use a synchronous call when the caller needs an immediate decision. I set a deadline, bound concurrency, and define what a timeout means. I avoid a long chain of dependent calls when it makes latency and failure hard to control."

Example: Validate a payment request and return whether it was accepted for processing. Do not promise final settlement if that occurs later.

Demonstration: `POST /payment-instructions` synchronously authenticates, validates, and commits an instruction record. It returns an ID and `ACCEPTED` once local acceptance is durable; settlement occurs asynchronously.

Why synchronous for acceptance: The caller must know whether its request was durably recorded before leaving the screen. **Proof:** A successful response always maps to a retrievable instruction, and a commit failure never produces a successful acknowledgment. **Limit:** Provider settlement should not be implied by the acceptance HTTP status. Long dependency chains can consume the caller's latency budget.

Code anchor (illustrative fragment):

```
HttpRequest request = HttpRequest.newBuilder(uri)
    .timeout(Duration.ofSeconds(2))
    .POST(BodyPublishers.ofString(payload))
    .build();
HttpResponse<String> response =
    client.send(request, BodyHandlers.ofString());
// Timeout is a caller deadline, not proof of remote failure.
```

Verify it: Verify request deadlines and distinguish accepted-for-processing from fully settled. Client and transport settings may add other timeouts.

6. Event-driven and publish/subscribe architecture [High]

Plain English: A service emits a fact about a completed change; other services respond when they receive it. This reduces direct coupling but introduces delivery delay and duplicate or out-of-order events.

Professional answer: "I use events when downstream consumers can react independently and eventual consistency is acceptable. I define event ownership, schema evolution, idempotent consumers, ordering needs, and how failed events are replayed."

Example: `PaymentAccepted` can trigger audit and status projection. Do not put sensitive data in an event merely for convenience. A command tells one component to do something; an event tells interested components something happened.

Demonstration: After a payment settles, the Payment service emits `PaymentSettled` with event ID, instruction ID, schema version, and non-sensitive status metadata. A reconciliation projection and an audit consumer update independently.

Why events: Payment does not need to wait for the two reactions to finish. **Proof:** Replay the event and verify that idempotent consumers do not duplicate their results; a delayed event shows an explicit 'last updated' time on the projection. **Limit:** An event is not a synchronous query or a guarantee of immediate read consistency.

Code anchor (illustrative fragment):

```
record PaymentAccepted(PaymentId id, UUID eventId) {}
void onPaymentAccepted(PaymentAccepted event) {
    if (!inbox.claimOnce(event.eventId())) return;
    auditProjection.apply(event.id());
}
// Claim and projection update need an atomic strategy.
```

Verify it: Replay an event and deliver it out of order. Decide whether ordering matters and how projection versioning handles it.

7. Transactional outbox [High]

Plain English: Saving business data and publishing an event are two writes. One can succeed while the other fails. An outbox saves the business change and event record in one local database transaction; a relay publishes the event afterward.

Professional answer: "I use an outbox when a committed state change must reliably produce an event. The local transaction writes both business and outbox rows. A relay may publish more than once, so consumers still need idempotency and monitoring."

Cost: Outbox storage, relay, lag, cleanup, ordering decisions. It does not make an external payment provider part of the local transaction. See AWS transactional outbox.

Demonstration — dual-write failure: If `savePayment()` commits but `broker.publish()` fails, downstream consumers never hear about it. Instead, within a **single database transaction**, insert the payment row and an outbox row containing `eventId`, type, and payload. A separate relay publishes pending records and marks or tracks delivery.

Why outbox: It closes the gap between committed local state and a missing event. **Proof:** Kill the process after DB commit and before publish: the relay still finds the event; publish twice and verify the consumer effect occurs once. **Limit:** The relay can redeliver, so it is **not** a blanket exactly-once guarantee. Monitor relay lag and retention.

Code anchor (illustrative fragment):

```
BEGIN;
INSERT INTO payment_instruction(id, status)
VALUES (:id, 'ACCEPTED');
INSERT INTO outbox(event_id, aggregate_id, payload)
VALUES (:event_id, :id, :payload);
COMMIT;
-- A separate relay publishes outbox rows later.
```

Verify it: Roll back either insert and confirm both rows disappear. Test duplicate relay publishing and durable consumer deduplication.

8. Saga [Medium; later design round]

Plain English: A workflow across several services is split into local steps. If a later step fails, the system performs a defined compensating action where possible. Compensation is a new business operation, not a magic database rollback.

Professional answer: "I consider a saga when one business process crosses independently owned services and a single database transaction cannot cover it. I define states, retry and compensation

behavior, and what happens after an irreversible step. For a simple workflow within one database, a local transaction is safer and easier."

Example: Reserve funds, submit instruction, record confirmation. If submission fails before an irreversible action, release the reservation. If the external outcome is unknown, reconcile before resubmitting. **Cost:** State tracking, delayed consistency, failed compensation and operational complexity. See Microsoft saga pattern.

Demonstration: A cross-service onboarding workflow creates a customer identity, provisions an account, and activates a payment profile. If account provisioning fails, the orchestrator requests an identity cleanup *where business rules allow*. If profile activation has already caused an irreversible outside action, the workflow needs a new reversal or review step, not a fake rollback.

Why saga: Each service commits to its own store, and no safe single ACID transaction covers the workflow. **Proof:** Test failure at each step, including a failed compensation, and ensure the saga ends in a visible `NEEDS_REVIEW` state instead of silently disappearing. **Avoid it:** One service and one local database transaction need no saga.

Code anchor (illustrative fragment):

```
void handle(ReservationSucceeded e) {
    saga.markReserved(e.orderId());
    clearing.submit(e.orderId());
}
void handle(SubmissionRejected e) {
    reservation.requestRelease(e.orderId());
    saga.markCompensating(e.orderId());
}
// Compensation is another action; it can fail.
```

Verify it: Simulate compensation failure and an unknown external outcome. Do not assume a remote charge can be rolled back.

9. CQRS [Medium; later design round]

Plain English: Commands change state; queries read it. CQRS separates their models so each can fit its job. This can be as small as separate code paths in one database or as large as a separate read store.

Professional answer: "I use CQRS only when read and write needs differ enough to justify separate models. For example, payment changes may enforce strict invariants while reconciliation dashboards need a denormalized view. I would define how the view is updated and what users see during lag."

Cost: More code and consistency management. CQRS does not require event sourcing, a message broker, or microservices. See Microsoft CQRS pattern.

Demonstration: A payment write path enforces approval and settlement invariants using normalized transactional tables. A reconciliation dashboard reads a denormalized projection keyed by tenant/date/status to avoid expensive joins for every chart.

Why CQRS: The write model optimizes correctness; the read model optimizes a genuinely different query shape. **Proof:** Compare dashboard latency and write throughput, and test how the UI displays a projection that is several seconds behind. **Limit:** CQRS can use one database and does not imply event sourcing; if one simple query meets the need, separation adds needless lag and operational work.

Code anchor (illustrative fragment):

```
// Write model owns invariants and state transitions.
void approve(ApprovePayment cmd) { /* validate + persist */ }

// Read model is optimized for a screen.
record PaymentSummary(String id, String status, Money total) {}
PaymentSummary dashboardRow(PaymentId id) { /* projection */ }
```

Verify it: Measure projection lag when using a separate read store. CQRS can also use one database and separate query models.

10. Event sourcing [Low for this round]

Plain English: Store a sequence of events as the source of truth and rebuild current state from them. An audit log beside a normal database is not automatically event sourcing.

Professional answer: "Event sourcing can provide a detailed history and support rebuilding projections, but it adds schema evolution, replay, ordering, and operational complexity. I would use it only for a clear requirement that outweighs those costs."

Example: Account ledger events may be valuable, but financial audit requirements alone do not force full event sourcing. Know snapshots and replay only at a conceptual level now.

Demonstration: Rather than storing only `account.balance = 500`, an event-sourced ledger stores an ordered history such as `AccountOpened`, `FundsCredited(700)`, and `FundsDebited(200)`; the current balance is derived by replaying the accepted events.

Why event sourcing: Reconstructing prior state and knowing *which accepted changes* produced it are primary requirements. **Proof:** Rebuild a projection from an empty state and compare it with the live one; test event-version upgrades and replay handling. **Cost:** Corrections become new events, projections lag, and historical schemas must remain interpretable. A table of audit records alongside current state is not event sourcing.

Code anchor (illustrative fragment):

```
record Deposited(Money amount) implements AccountEvent {}
record Withdrawn(Money amount) implements AccountEvent {}
Balance replay(List<AccountEvent> events) {
    Balance balance = Balance.zero();
    for (AccountEvent e : events) balance = apply(balance, e);
    return balance;
}
// Current balance derives from stored events.
```

Verify it: Replay event history and compare the final state. Audit logs next to mutable balances are not automatically event sourcing.

11. Cache-aside [High]

Plain English: Read the cache first. On a miss, read the database and fill the cache. When source data changes, invalidate or update the cached copy according to a freshness policy.

Professional answer: "I use cache-aside for repeated reads where stale data is acceptable within a defined bound. I set tenant-aware keys, TTLs, invalidation rules, and stampede protection, then measure hit rate and tail latency. I keep authoritative payment state and permissions on a deliberate consistency path."

Cost: Staleness, extra failure mode, memory, invalidation races. A 35% latency improvement is useful only when the baseline and workload are known. See Redis cache-aside guidance.

Demonstration: A public currency-metadata lookup reads `cache[tenant+currency]`; on a miss it reads the authoritative store and fills the cache. On a metadata update, invalidate the affected key. The key must include every dimension that changes the answer.

Why cache-aside: Repeated reads can be served faster when a defined staleness window is tolerable. **Proof:** Measure hit rate and p95 latency under a representative workload; test concurrent miss storms and invalidation races. **Limit:** Do not treat cached balance, mutable payment status, or authorization as authoritative without a separately justified consistency design.

Code anchor (illustrative fragment):

```
PaymentView get(PaymentId id) {
    PaymentView cached = cache.get(id);
    if (cached != null) return cached;
    PaymentView fresh = repository.readView(id);
    cache.put(id, fresh, ttl);
    return fresh;
}
// Writes must invalidate/update this view deliberately.
```

Verify it: Test a cache miss, a hit, and staleness after a write. Use tenant-aware keys and exclude unsafe permission shortcuts.

12. Circuit breaker, retry, timeout, and bulkhead [High]

Plain English: A timeout limits waiting. A retry tries again when failure may be temporary and repetition is safe. A circuit breaker stops calls to a dependency that keeps failing. A bulkhead limits how much of the application's resources that dependency can consume.

Professional answer: "I combine bounded timeouts, selective retries, a circuit breaker, and separate resource limits according to the dependency's failure behavior. I also define a safe fallback or clear error. For payments, a timeout is an uncertain outcome, so idempotency and reconciliation must be part of retry design."

Cost: A retry storm can make an outage worse; a circuit breaker can reject a recovered service until probing succeeds; bulkheads use extra capacity. Tune from latency and error data. See Microsoft circuit breaker and bulkhead.

Demonstration — provider slows down: Set a per-attempt deadline so calls do not wait forever. Retry only safe transient errors with bounded attempts and jitter. If failure rates stay high, open a circuit breaker to fail fast. Give provider calls a separate small concurrency pool so an outage does not exhaust threads needed for status APIs.

Why all four: Timeout bounds a call; retry handles temporary failure; breaker avoids repeated work against an unhealthy dependency; bulkhead limits the blast radius. **Proof:** In a test outage, status queries still work, calls are capped, and request traffic does not grow into a retry storm. **Payment caution:** A timed-out submission has an **unknown** outcome until deduplication/reconciliation proves otherwise.

Code anchor (illustrative fragment):

```
// Pseudocode: actual library API and ordering vary.
withBulkhead("clearing", () ->
    withCircuitBreaker("clearing", () ->
        retryOnlyIfSafe(() ->
            callWithDeadline(provider::submit))));
// A timeout does not prove whether provider acted.
```

Verify it: Inject high latency and persistent failures. Verify resource isolation and bounded attempts; prevent duplicate charges.

13. Queue-based load leveling [Medium]

Plain English: Put work into a durable queue when producers can send faster than consumers can safely process. Consumers take work at a controlled rate.

Professional answer: "I use a queue when the caller does not require immediate completion and burst traffic would overload a dependency. I define capacity, acceptance response, retry, duplicate handling, dead-letter treatment, and backlog monitoring."

Cost: Delayed results, eventual consistency, operational queues, ordering limitations. A queue does not remove a bottleneck; it changes how you absorb and expose it. See Microsoft load-leveling pattern.

Demonstration: A partner sends a burst of 30,000 import records, while the clearing integration can process only a smaller controlled rate. The API durably enqueues validated work and returns a job ID; consumers drain the queue at the downstream-safe rate.

Why queueing: It absorbs bursts without requiring the slower dependency to match peak arrival rate. **Proof:** During the burst, queue depth and oldest-message age rise predictably and later return to normal; duplicate deliveries do not duplicate payment effects. **Cost:** The queue does not increase steady-state provider capacity. Set backlog limits, retention, poison-message routing, and an honest `PENDING` status.

Code anchor (illustrative fragment):

```
@PostMapping("/reports")
ResponseEntity<?> accept(ReportRequest request) {
    JobId id = jobs.persist(request); // durable first
    queue.enqueue(id);
    return ResponseEntity.accepted().body(id);
}
// Worker processes at bounded concurrency.
```

Verify it: Burst-load the endpoint and confirm stable worker concurrency. Handle the DB-plus-queue dual write with an outbox or equivalent.

14. Strangler Fig migration [Medium]

Plain English: Replace a legacy system a piece at a time. Route selected capabilities through new code while the old system continues handling the rest.

Professional answer: "For a risky modernization, I favor a controlled migration by capability or customer segment. I define routing, data ownership, reconciliation between old and new behavior, rollback, and a retirement plan for the legacy path."

Example: Move one payment type or client group from file-based processing to a new service, compare results, expand traffic gradually, then remove the old path. **Cost:** Temporary duplication and complex coexistence. See Martin Fowler's Strangler Fig pattern.

Demonstration: During a file-to-API migration, route one well-defined payment category to the new path, keep the others on the legacy path, and compare submitted totals and reconciliation outcomes by route. Increase traffic only after parity and failure criteria are met.

Why Strangler Fig: It replaces a capability incrementally instead of risking a full cutover. **Proof:** A routing flag can redirect *new* eligible work to the legacy path without moving already-processing transactions; reconciliation finds no missing or double-submitted instructions. **Tradeoff:** Temporary dual operations, schema translation, observability, and a clear retirement plan are necessary.

Code anchor (illustrative fragment):

```
PaymentResult submit(Payment p) {
    if (routing.usesNewProvider(p.customerId()))
        return newGateway.submit(p);
    return legacyGateway.submit(p);
}
// Route each instruction once; preserve its chosen path.
```

Verify it: Compare reconciliation outcomes by cohort. Rollback routing for *new* work only; avoid submitting an in-flight instruction twice.